Security Review

Cantina Managed

COINBASE: 200MS BLOCKS

Cantina Managed review by:
Slowfi, Lead Security Researcher
Jay, Security Researcher

October 1, 2026

Contents

1 Introduction **2**
- 1.1 About Cantina . . . 2
- 1.2 Disclaimer . . . 2
- 1.3 Risk assessment . . . 2
  - 1.3.1 Severity Classification . . . 2

2 Security Review Summary **3**
- 2.1 Scope . . . 3

3 Findings **4**
- 3.1 Low Risk . . . 4
  - 3.1.1 Parent system-config lookup can return stale configuration after a reorg . . . 4
  - 3.1.2 Stale ancestry lookups can change Denim channel validation after a reorg . . . 4
  - 3.1.3 Default proof history covers three days at a sustained Denim cadence . . . 5
- 3.2 Informational . . . 5
  - 3.2.1 Off by one in the per height block retention cap . . . 5
  - 3.2.2 Inconsistent integer overflow handling across the three millisecond timestamp computations . . . 6
  - 3.2.3 Sequencer and verifier disagree on a zero activation timestamp . . . 6
  - 3.2.4 Onchain intermediate interval is not pinned to the ZK prover sampling cadence . . . 7
  - 3.2.5 Speedup activation trusts schedule index 13 without binding it to Denim . . . 7
  - 3.2.6 BaseTime predeploy activation does not pin the proxy runtime to its canonical code hash . . . 8
  - 3.2.7 Staged sync does not enforce the Denim timestamp schedule . . . 8
  - 3.2.8 An additional BaseTime update can diverge from the claimed block time . . . 9
  - 3.2.9 BaseTime metadata can be accepted before Denim activation . . . 9
  - 3.2.10 The first transaction can read the previous Denim time . . . 9
  - 3.2.11 Pending transaction millisecond metadata depended on request order . . . 10
  - 3.2.12 Queued reorgs can overwrite an earlier proof history unwind . . . 10

1 Introduction

1.1 About Cantina

Cantina is a security platform that pairs a world-class network of security researchers with agentic AI to help teams building onchain find and fix real risk. Through managed reviews, competitions, and bounties, Cantina secures protocols before vulnerabilities can be exploited.
Learn more at cantina.security

1.2 Disclaimer

Cantina Managed provides a detailed evaluation of the security posture of the code at a particular moment based on the information available at the time of the review. While Cantina Managed endeavors to identify and disclose all potential security issues, it cannot guarantee that every vulnerability will be detected or that the code will be entirely secure against all possible attacks. The assessment is conducted based on the specific commit and version of the code provided. Any subsequent modifications to the code may introduce new vulnerabilities that were absent during the initial review. Therefore, any changes made to the code require a new security review to ensure that the code remains secure. Please be advised that the Cantina Managed security review is not a replacement for continuous security measures such as penetration testing, vulnerability scanning, and regular code reviews.

1.3 Risk assessment

Severity level	Impact: High	Impact: Medium	Impact: Low
Likelihood: high	Critical	High	Medium
Likelihood: medium	High	Medium	Low
Likelihood: low	Medium	Low	Low

1.3.1 Severity Classification

The severity of security issues found during the security review is categorized based on the above table. Critical findings have a high likelihood of being exploited and must be addressed immediately. High findings are almost certain to occur, easy to perform, or not easy but highly incentivized thus must be fixed as soon as possible.

Medium findings are conditionally possible or incentivized but are still relatively likely to occur and should be addressed. Low findings are a rare combination of circumstances to exploit, or offer little to no incentive to exploit but are recommended to be addressed.

Lastly, some findings might represent objective improvements that should be addressed but do not impact the project's overall security (Gas and Informational findings).

2 Security Review Summary

Base is a secure, low-cost, builder-friendly Ethereum L2 built to bring the next billion users onchain.

From Sep 22nd to Sep 25th the Cantina team conducted a review of base, base and contracts on commit hashes 48786432, 26902ae0 and 530f49a5 respectively. The team identified a total of **15** issues:

Issues Found

Severity	Count	Fixed	Acknowledged
Critical Risk	0	0	0
High Risk	0	0	0
Medium Risk	0	0	0
Low Risk	3	3	0
Gas Optimizations	0	0	0
Informational	12	4	8
Total	**15**	**7**	**8**

2.1 Scope

The security review had the following components in scope for base, base and contracts on commit hashes 48786432, 26902ae0 and 530f49a5 respectively:

```
├── base
│   └── crates
│       ├── common
│       │   ├── chains
│       │   │   └── src
│       │   │       └── upgrade.rs
│       │   ├── evm
│       │   │   └── src
│       │   │       ├── base_time.rs
│       │   │       └── executor
│       │   │           └── block_executor.rs
│       │   ├── evm2
│       │   │   └── src
│       │   │       ├── base_time.rs
│       │   │       ├── executor.rs
│       │   │       └── transition.rs
│       │   └── genesis
│       │       └── src
│       │           └── rollup.rs
│       ├── consensus
│       │   ├── derive
│       │   │   └── src
│       │   │       └── attributes
│       │   │           └── stateful.rs
│       │   ├── gossip
│       │   │   └── src
│       │   │       └── block_validity.rs
│       │   └── protocol
│       │       └── src
│       │           ├── base_time.rs
│       │           └── batch
│       │               └── span.rs
│       └── execution
│           ├── consensus
```

```
│           │   └── src
│           │       └── validation
│           │           └── mod.rs
│           └── rpc
│               └── src
│                   └── eth
│                       └── base_time.rs
└── contracts
    └── src
        ├── L1
        │   └── proofs
        │       └── AggregateVerifier.sol
        └── L2
            └── BaseTime.sol
```

3 Findings

3.1 Low Risk

3.1.1 Parent system-config lookup can return stale configuration after a reorg

Severity: Low Risk.

Context: stateful.rs#L106.

Description: `prepare_payload_attributes` in `stateful.rs` fetches the parent system configuration using `system_config_by_number`, discarding the parent hash already available in `l2_parent`. The underlying provider caches blocks by number without invalidating those entries on reorg, so the lookup can return a block from the old branch rather than the parent the builder is actually extending.

If a reorg replaces a cached block with another block at the same height carrying different configuration, the builder can use stale gas limits or fee parameters when constructing payload attributes. The refresh logic immediately above this call forces this lookup around Denim activation, making correct parent identification particularly relevant at the upgrade boundary. Block numbers were already ambiguous across reorgs before Denim; the `200` ms cadence does not introduce that ambiguity.

The potential consequence is payload construction using configuration inconsistent with the selected parent, which could disrupt derivation or cause disagreement with nodes using the correct configuration. Full-node disruption or a chain split has not been demonstrated. Optimism addressed the same lookup pattern by retaining the parent hash when requesting system configuration: Optimism PR 22465.

Recommendation: Fetch the parent system configuration using `l2_parent.block_info.hash` and cache the result by hash. This binds the returned configuration to the exact parent being extended, regardless of changes to the canonical block at that height.

Coinbase: Fixed in PR 5274.

Cantina Managed: Fix verified.

3.1.2 Stale ancestry lookups can change Denim channel validation after a reorg

Severity: Low Risk.

Context: batch_validator.rs#L262.

Description: The same-second ancestry check in `batch_validator.rs` calls `l2_block_info_by_number` to determine whether a batch's claimed parent matches a recent canonical ancestor. A match causes the validator to skip that batch while preserving the remaining channel. An invalid parent instead causes the remaining channel to be discarded.

The underlying `AlloyL2ChainProvider` lookup uses a number-keyed cache without revalidating cached entries after a reorg. If a block at a cached height is replaced, the lookup can return the old branch's block. Nodes with different cache contents could therefore classify the same batch differently: one could skip only that batch while another discards the remaining channel.

Recommendation: Ensure ancestry validation cannot consume stale number-cache entries. Fetch current canonical block information directly, or anchor the ancestry walk to the supplied safe head and follow its parent hashes backward.

The hash-based approach must start from the trusted safe head. Fetching the untrusted batch's claimed parent by hash alone does not establish canonical ancestry.

Add regression coverage verifying that stale cached blocks cannot change the skip-versus-flush decision.

Coinbase: Fixed in PR 5361. The specific stale-cache path is fixed on current main: `l2_block_info_by_number` now bypasses the number cache and fetches current canonical block information.

Cantina Managed: PR 5361 proposes additional hardening by anchoring ancestry to the safe head; it remained open and unmerged when checked. Production deployment has not been verified.

3.1.3 Default proof history covers three days at a sustained Denim cadence

Severity: Low Risk.

Context: args.rs#L16-L17, snapshots.rs#L16.

Description: The reviewed proof-history ExEx and manual prune command defaulted to a window of `1,296,000` blocks. At Denim's sustained rate of five blocks per second, this retains three days, despite the code describing it as a month at the old two-second cadence. The independent optional `--full` and full-snapshot preset retained `1,339,200` blocks of bodies, receipts, and state history, a configured distance of about `3.1` days at the Denim rate.

The dispute verifier can leave a one-proof game open for at least five days, while proof and witness tooling can request historical state from L2 nodes. An operator relying only on either affected retention setting, without an archive source or previously captured witness, could lack data needed for a late proof. This is a conditional availability risk, not a demonstrated failed deployed dispute or a consensus fault. Proof history and `--full` are independently enabled, and their deployed configurations were not established.

Recommendation: Set each retention default from the longest supported wall-clock proof workflow and the fastest supported block cadence, allowing for game creation lag and retries. Keep archive or captured-witness access for history already pruned, and test historical proof and witness retrieval after the configured pruning boundary.

Coinbase: Fixed in PR 5362 and PR 5367.

Cantina Managed: PR 5362 was merged as commit f76113c5, and PR 5367 was merged as commit 90291981. The proof-history ExEx, node CLI, and manual prune default now use `6,480,000` blocks, or `15` days at five blocks per second. The separate `--full` preset now uses `13,392,000` blocks, or `31` days. The current main branch retains both values, and the merged build, unit-test, and system-test checks passed. Static review found no distinct code regression from these default changes. They do not restore previously pruned data, prevent an operator from choosing a shorter override, or prove that every possible late challenge fits within `15` days; a live pruning-to-dispute test was not run.

3.2 Informational

3.2.1 Off by one in the per height block retention cap

Severity: Informational.

Context: block_validity.rs#L232-L234.

Description: `BlockHandler::MAX_BLOCKS_TO_KEEP` is set to `5` and documented as the maximum number of distinct blocks to keep per height. The enforcement check uses a strict greater than comparison, and the length is evaluated before the new hash is inserted. As a result a single height can hold `6` distinct hashes before the 7th is rejected, so the effective cap is `6` rather than the documented `5`.

This bound exists to limit how many distinct block hashes are tracked per height, which protects against unbounded memory growth from gossip spam. A cap of `6` instead of `5` is a negligible difference in that resource bound and is not exploitable. The behavior also matches the op node reference implementation, so it reflects intended cross client parity rather than a Denim regression. Using a greater than or equal comparison would enforce a true cap of `5`.

Recommendation: If a hard cap of `5` is desired, change the comparison to greater than or equal so the count is enforced before exceeding the documented limit. If parity with the op node reference is the priority, keep the current behavior and update the documentation to state that up to `6` distinct hashes may be retained per height.

Coinbase: Fixed in PR 5269.

Cantina Managed: Fix verified.

3.2.2 Inconsistent integer overflow handling across the three millisecond timestamp computations

Severity: Informational.

Context: base_time.rs#L106-L107.

Description: The millisecond L2 block timestamp is computed in three places, each with a different overflow behavior for the same value.

- RPC path, `BaseTimeUpdateTx::extract_timestamp_ms`: `wrapping_mul` then `wrapping_add`, so overflow silently wraps to a small value.
- Consensus path, `RollupConfig::l2_block_timestamp_millis`: saturating arithmetic, so overflow clamps to `u64::MAX`.
- Solidity predeploy, `BaseTime.timestampMs`: `uint64(block.timestamp * 1000 + timestampMillisPart)`, so the `uint64` cast truncates modulo `2^64`.

For every realistically reachable input the three paths return the identical value. They diverge only once the seconds denominated timestamp exceeds `u64::MAX / 1000`, roughly `1.8e16` seconds or about `585` million years past epoch, a regime consensus bounds make unreachable.

The RPC path is read only and feeds cached timestamps into RPC responses, so the divergence has no reachable safety, liveness, or economic impact. The concern is consistency and defensive coding. Three overflow behaviors for one value is a latent footgun if the surrounding bounds ever change.

Recommendation: Align the three computations on a single overflow behavior across the RPC path, the consensus path, and the Solidity predeploy. Prefer plain checked arithmetic so an overflow that should never occur surfaces as a hard failure rather than being masked. Matching the saturating behavior of `l2_block_timestamp_millis` is an acceptable interim step.

Coinbase: Acknowledged.

Cantina Managed: Acknowledged.

3.2.3 Sequencer and verifier disagree on a zero activation timestamp

Severity: Informational.

Context: AggregateVerifier.sol#L1229.

Description: The fast block speedup activation is read from the same schedule value on two sides, but the two sides interpret a value of zero differently.

On the contract side, `_firstFastBlock` treats a schedule value of zero as unset and returns the never sentinel `type(uint256).max`. The verifier therefore stays on the slow cadence forever.

On the node side, `denim_activation_block_number` only returns `None` when the schedule key is absent. An explicit value of zero returns `Some(0)`, which resolves to activation offset zero, so the sequencer runs the fast cadence from genesis.

Because an absent key is `None` but an explicit zero is a set value, and zero means from genesis elsewhere in the configs, a chain configured with the activation timestamp set to zero would split the sequencer and the verifier on the block to timestamp mapping. The sequencer would produce fast cadence blocks from genesis while the verifier would derive timestamps on the slow cadence, so honest proposals would fail verification.

This is not reachable on mainnet, where the speedup is scheduled at a real nonzero timestamp, and the team avoids a zero activation even on devnet. The impact is confined to a misconfiguration, but the two implementations do not agree on what not scheduled means.

Recommendation: Normalize a zero activation timestamp to the unset value at config load on the node so both sides agree, or reject a zero activation timestamp during config validation. This closes the divergence at the source rather than relying on operational discipline to never use zero.

Coinbase: Acknowledged.

Cantina Managed: Acknowledged.

3.2.4 Onchain intermediate interval is not pinned to the ZK prover sampling cadence

Severity: Informational.

Context: AggregateVerifier.sol#L1207.

Description: The `AggregateVerifier` constructor validates the proposal intervals through `_validateIntervals`. The only cross check it performs is that the slow and fast interval pairs yield the same intermediate root count, at `AggregateVerifier.sol:1207`, because that count freezes the CWIA `extraData` layout for the lifetime of the implementation.

Nothing constrains the intermediate block interval itself to any specific value. It is a free deploy parameter stored as an immutable. `intermediateOutputRootsCount` and the challenge range built in `_getStartingIntermediateRootAndL2SequenceNumbers` are both derived from that interval dynamically.

The offchain ZK prover samples intermediate roots at a hardcoded cadence of `30` blocks, and the per game interval parameter that would let it honor the deployed value was dropped. Because the onchain interval and the prover cadence are set independently, a deploy that configures a different intermediate interval produces a root count and a challenge range that the prover does not match. Every ZK proof for that cadence then fails verification.

The one committed config in the repo, `deploy-config/local.json`, already sets these to `10` and `100` rather than `30`, though it is a local development config that may run against a mock verifier and mask the mismatch in tests.

Recommendation: Make the onchain intermediate interval and the prover sampling cadence provably agree, in either direction. Either pin the onchain intervals to the prover constant so a mismatched config reverts at deploy time, or make the prover dynamic so it reads the deployed interval and samples accordingly. The second option preserves a configurable interval and is the stronger long term fix. In both cases the goal is that a configuration the prover cannot satisfy is rejected loudly rather than failing silently later.

Coinbase: Acknowledged.

Cantina Managed: Acknowledged.

3.2.5 Speedup activation trusts schedule index 13 without binding it to Denim

Severity: Informational.

Context: AggregateVerifier.sol#L1228.

Description: `_firstFastBlock` resolves the fast block activation timestamp by reading a single hardcoded position in the protocol versions schedule. That position is the constant `FAST_BLOCK_UPGRADE_INDEX` which is set to `13`, and nothing on this path binds the slot to Denim by name or by hash. The verifier simply trusts that whatever value sits at index `13` is the Denim activation timestamp. If another upgrade were ever registered at index `13` before Denim, the verifier would silently read the wrong activation timestamp and compute an incorrect slow to fast block mapping with no error raised. Beyond the index itself, nothing here confirms that this timestamp, or the genesis timestamp and block time immutables, agree with the node config, so the entire correspondence between the contract and the node rests on deployment discipline rather than an enforced invariant. In practice this is not exploitable. Hard forks are scheduled one after another so index `13` will hold Denim on any correctly ordered deployment, and mainnet schedules Denim at a real slot. The concern is that a schedule layout change or a deployment mistake would fail quietly rather than loudly.

Recommendation: Add a deploy time assertion that the schedule slot really corresponds to Denim, for example by checking the schedule identifier for that index against an expected value, and add a runbook check that the contract immutables match the node config. This ensures the contract and the node cannot drift apart silently if the schedule layout ever changes.

Coinbase: Acknowledged.

Cantina Managed: Acknowledged.

3.2.6 BaseTime predeploy activation does not pin the proxy runtime to its canonical code hash

Severity: Informational.

Context: base_time.rs#L124-L125.

Description: The Denim predeploy activation in `ensure_predeploy` links the `BaseTime` implementation only after checking that the proxy account has non empty code and that its EIP-1967 admin slot equals the expected proxy admin. That is the full extent of the validation. It confirms that some contract exists at the `BaseTime` address and that the admin slot looks correct, but it never verifies that the deployed runtime is actually the canonical predeploy proxy.

The constant `PROXY_CODE_HASH` exists for exactly this purpose, yet it is only referenced from a unit test and never from the activation path. As a result the runtime bytecode behind the proxy is never compared against the pinned value, so a proxy that carries the expected admin slot but a different runtime would still pass activation and have the implementation linked behind it.

Recommendation: Consider verifying the proxy runtime against an expected code hash before linking the implementation. Where legitimate proxy runtimes differ between chains, use a chain-specific or explicitly configured expected hash. Alternatively, verify this invariant during genesis validation and document it as an activation prerequisite.

Coinbase: Acknowledged.

Cantina Managed: Acknowledged.

3.2.7 Staged sync does not enforce the Denim timestamp schedule

Severity: Informational.

Context: mod.rs#L27-L38.

Description: The Base execution consensus check validates the format of the `BaseTime` deposit at `tx[1]`, but does not compare its millisecond value with the schedule for that block number. Staged backfill uses this check without the Engine post execution progression hook. In a synthetic full node test, an operator selected a malformed tip whose child claimed `.400` when the configured schedule required `.200`. The target completed staged backfill and persisted that child and its `BaseTime` state, while the CL schedule helper rejected it.

An operator selected malformed target can therefore be persisted despite conflicting with the CL schedule. The test does not show that an untrusted peer can choose the target or that normal CL gossip or safe derivation accepts the block. No public trigger or safe chain effect was demonstrated.

Recommendation: Apply the configured absolute Denim timestamp schedule to staged imports as well as the Engine and locally built insertion paths. Test a valid parent followed by a child whose correctly formatted `BaseTime` deposit contains an off schedule millisecond value.

Coinbase: Fixed in PR 5332, PR 5334, and PR 5335.

Cantina Managed: Verified the complete proposed stack at commit f01fdfe. Consensus and execution now share the timestamp schedule. At Denim heights, header validation checks scheduled seconds, and body, pre execution, and staged post execution validation check the full timestamp and `BaseTime` metadata. The cached insertion wrapper applies the same check before delegating insertion of a locally executed block. The four relevant crate suites passed `433` tests, and targeted

checks confirmed rejection of the reported `.400` child where `.200` was required, incorrect activation seconds, and missing metadata in staged post execution validation.

3.2.8 An additional BaseTime update can diverge from the claimed block time

Severity: Informational.

Context: base_time.rs#L87-L96.

Description: The validator extracts the mandatory `BaseTime` deposit from `tx[1]` but does not reject another depositor authorized setter later in the block. A privileged unsafe producer can supply the scheduled `.200` update at `tx[1]` and a `.400` update at `tx[2]`. In a synthetic signed payload test, CL gossip checks accepted the envelope, a separate cold Engine accepted the same payload bytes, and the selected state stored `.400` while block RPC reported the `.200` claim. The CL insertion test used a mocked Engine response; normal safe derivation does not construct this duplicate.

Such an unsafe block can expose a different executed clock from its metadata claim and may need reorganization before normal progression. This requires a faulty or malicious privileged producer. An ordinary user cannot produce the necessary protocol deposit.

Recommendation: Consider rejecting additional `BaseTime` setter deposits outside `tx[1]` and checking the final executed `BaseTime` state against the accepted metadata. Test both the duplicate setter and a correctly formatted setter that does not update storage.

Coinbase: Acknowledged.

Cantina Managed: Acknowledged.

3.2.9 BaseTime metadata can be accepted before Denim activation

Severity: Informational.

Context: mod.rs#L27-L38, base_time.rs#L59-L71.

Description: The execution consensus helper requires `BaseTime` metadata after Denim activation but does not reject a privileged `BaseTime` setter before activation. In a synthetic chain with the `BaseTime` implementation already linked, a correctly timed pre Denim block with the setter at `tx[1]` was accepted by a cold Engine importer and selected by forkchoice. RPC reported a millisecond timestamp derived from that deposit even though the legacy schedule has no subsecond slot. A setter at a later position can also execute without appearing in this RPC field.

A privileged unsafe branch can therefore expose or execute premature `BaseTime` values. Normal safe derivation does not create these transactions, and the demonstrated state change depends on a privileged producer and synthetic preactivation state. This is not a public deposit attack or a demonstrated safe chain violation.

Recommendation: Consider rejecting depositor authorized `BaseTime` setters before activation without rejecting unrelated deposits, and gate RPC millisecond extraction on activation. Test setters at `tx[1]` and later positions with both linked and dormant predeploy states.

Coinbase: Acknowledged.

Cantina Managed: Acknowledged.

3.2.10 The first transaction can read the previous Denim time

Severity: Informational.

Context: mod.rs#L27-L38.

Description: The execution consensus check validates the `BaseTime` deposit at `tx[1]` but does not constrain the recipient of the L1 info deposit at `tx[0]`. A privileged unsafe signer can redirect otherwise decodable L1 info calldata to a reader contract. That deposit executes before the valid `BaseTime` update, so the reader can observe the previous millisecond value while final state and metadata agree on the new value. A synthetic test passed signed gossip checks and cold Engine execution for the same payload bytes, with a mocked Engine response in the CL insertion portion.

A contract reached by this forged first deposit can observe stale time within an unsafe block. This requires privileged payload production; an ordinary user cannot use this route. Normal safe derivation creates the canonical first deposit and is not shown to preserve the forged branch.

Recommendation: Consider validating the canonical identity and recipient of `tx[0]` at a shared block boundary, or explicitly document reliance on the privileged producer. Test a redirected first deposit followed by a valid `BaseTime` update through gossip and Engine execution.

Coinbase: Acknowledged.

Cantina Managed: Acknowledged.

3.2.11 Pending transaction millisecond metadata depended on request order

Severity: Informational.

Context: base_time.rs#L44-L56.

Description: At the review baseline, an indexed request for an executed pending transaction could omit `blockTimestampMs` on a cold `BaseTime` cache because the canonical transaction lookup could not find the pending body. Requesting the pending block first warmed the cache, causing a later indexed request for the same transaction to include the field. Thus the same pending transaction could expose different millisecond metadata depending on request order. This was an RPC consistency issue with no demonstrated security impact.

Recommendation: Ensure that an indexed request can derive `blockTimestampMs` from the matching executed pending block when its body is not yet canonical, so the result does not depend on a previous RPC request warming the cache. Add a regression test comparing cold and warm requests for the same pending transaction before forkchoice.

Coinbase: Fixed in PR 5287.

Cantina Managed: In merge commit 74a922c, `BaseTimeCache::get` falls back to `pending_block()` only when its hash matches the requested block. The added test checks that cold and warm indexed pending requests return the same timestamp metadata before forkchoice. The main CI test job passed. The merged test was not rerun locally.

3.2.12 Queued reorgs can overwrite an earlier proof history unwind

Severity: Informational.

Context: sync_target.rs#L51-L60.

Description: With the optional `--proofs-history` ExEx enabled, `SyncTargetState::apply_next` replaced a pending unwind with the newest one. If an A-to-B reorg beginning at block `5` was followed by a B-to-C reorg beginning at block `9` before the storage worker consumed either action, the worker unwound only from block `9`. Old A-branch history at blocks `5` through `8` remained, while proof storage advanced its latest marker to the canonical C tip. This did not change the execution node's canonical chain, but it could cause historical `eth_getProof` responses to disagree with canonical state.

A two-node reproduction forced this notification overlap through the running ExEx worker and MDBX proof storage. All five affected `eth_getProof` responses returned the old storage value and had account proofs that failed verification against their canonical headers. Three non-overlapping controls returned valid proofs. A separate storage-level reproduction also showed stale rows on RocksDB,

although the live RPC reproduction did not use RocksDB. The overlap was deliberately scheduled; its occurrence rate in production was not measured. See the full-node reproduction.

Recommendation: When combining queued reorg actions, retain the earliest required unwind height and its original parent anchor while updating the forward target to the latest canonical tip. Test overlapping notifications on both storage backends and verify historical proofs against canonical header roots.

Coinbase: Fixed in PR 5363.

Cantina Managed: PR 5363 was merged as commit da999ae2. The changed state transition retains the earlier complete `BlockWithParent` when a later pending revert begins higher, and its regression test covers the reported `5`-then-`9` sequence. The current main branch retains this logic. The saved full-node control with an equivalent correction returned five valid proofs from the same payloads; the exact merged PR binary was not rerun locally. The merged build, unit-test, and system-test checks passed. No distinct regression was identified in the changed merge logic; unforced occurrence and RocksDB RPC behavior remain unmeasured.